



# ADMINISTRACIÓN DE SERVIDORES

DOCUMENTACIÓN SERVIDOR CORREO POSTFIX

## PROYECTO GRUPAL

José Luis Olmo Barberá  
Victor Manuel Vidal Molina  
Alvaro Griñón Martínez

**CURSO 2024/25**

## Índice

|                                                                                |          |
|--------------------------------------------------------------------------------|----------|
| <b>1. Explicación contenido añadido a la infraestructura</b>                   | <b>2</b> |
| 1.1. Código del contenedor postfix_mail en el docker-compose.yml . . . . .     | 3        |
| 1.2. Especificaciones técnicas . . . . .                                       | 3        |
| 1.3. Código del dockerfile usado para crear una imagen personalizada . . . . . | 4        |
| 1.4. Código del script 'entrypoint.sh' usado en el Dockerfile . . . . .        | 5        |
| 1.5. Explicación dockerfile y entrypoint . . . . .                             | 6        |
| <b>2. Pruebas de funcionamiento</b>                                            | <b>7</b> |
| 2.1. Comprobaciones dentro del contenedor: . . . . .                           | 7        |
| 2.2. Enviar correo: . . . . .                                                  | 8        |

## 1. Explicación contenido añadido a la infraestructura

Durante esta fase de implementación, se han incorporado un nuevo contenedor a la infraestructura anterior. A continuación se detallan las características y configuraciones implementadas en el contenedor en el área production:

- **Servidor Postfix\_mail:**

Los requisitos del contenedor son los siguientes:

- La empresa de creación de Software necesita un sistema de mensajería interno mediante correo electrónico. Se pide instalar y configurar un servidor Postfix.
- El servidor deberá instalarse en la red de producción.
- Realizar informe.

### 1.1. Código del contenedor postfix\_mail en el docker-compose.yml

```
# Bloque POSTFIX-MAIL en production
postfix_mail:
  build: ./production/postfix_mail
  container_name: postfix_mail
  restart: always
  cap_add:
    - NET_ADMIN
  dns:
    - 172.20.0.6
  networks:
    bridge_local: {}
    production_net:
      ipv4_address: 172.30.0.15
  volumes:
    - postfix_maildata:/var/mail
```

Figura 1: Contenedor postfix\_mail.

### 1.2. Especificaciones técnicas

- Nombre del contenedor: postfix\_mail.
- Reinicio automático si se detiene por error.
- NET\_ADMIN otorga privilegios administrativos de red al contenedor, necesario para ejecutar el contenido del entrypoint correctamente.
- IP asignada: 172.30.0.15.
- DNS: 172.20.0.6.
- Puertos: '2525:25'.
- Redes: production\_net que lo conecta a la infraestructura y bridge\_local que conecta el host con el contenedor.
- Volúmenes: permiten la persistencia de los datos.

### 1.3. Código del dockerfile usado para crear una imagen personalizada

```
# postfix_mail /Dockerfile
FROM debian:bullseye

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && \
    apt-get install -y postfix mailutils bsd-mailx rsyslog sudo nano net-tools iputils-ping dnsutils iproute2 && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

RUN useradd -m john && echo "john:john" | chpasswd && adduser john sudo

COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

CMD ["/entrypoint.sh"]
```

Figura 2: Dockerfile para crear imagen personalizada de postfix\_mail.

#### 1.4. Código del script 'entrypoint.sh' usado en el Dockerfile

```
#!/bin/bash

# Configurar gateway para producción
ip route del default 2>/dev/null
iface_prod=$(ip -o -4 addr show | grep 172\.\.30\. | awk '{print $2}' | head -n1)
if [[ -n "$iface_prod" ]]; then
    echo "Asignando gateway 172.30.0.1 a través de $iface_prod"
    ip route add default via 172.30.0.1 dev "$iface_prod"
fi

# DNS interno
echo "nameserver 172.20.0.6" > /etc/resolv.conf

#Generar main.cf
cat <<EOF > /etc/postfix/main.cf
myhostname = mail.production.com
mydomain = production.com
myorigin = \${myhostname}
inet_interfaces = all
inet_protocols = ipv4
mydestination = \${myhostname}, localhost.\${mydomain}, localhost
mynetworks = 127.0.0.0/8, 172.30.0.0/24"
smtpd_banner = \${myhostname} ESMTP
EOF

# Crear alias para usuarios locales
touch /etc/aliases
newaliases

# Crear buzones si no existen
mkdir -p /var/mail
touch /var/mail/root /var/mail/john
chown root:mail /var/mail/root
chown john:mail /var/mail/john
chmod 660 /var/mail/*

# Arranca postfix en foreground
service postfix start

# Mantiene el contenedor activo
tail -f /dev/null
```

Figura 3: Contenido script ejecutado en el dockerfile de postfix\_mail.

## 1.5. Explicación dockerfile y entryptoint

### ■ Dockerfile:

- Actualizamos e instalamos los paquetes necesarios en el contenedor, en este caso los esenciales son postfix y mutt.
- Creamos el usuario John y damos permisos, que aunque no se especificaba nada, al igual que en la práctica anterior lo hemos añadido a todos los contenedores.
- Copiamos los archivos necesarios desde fuera del contenedor hacia dentro y otorgamos permisos de ejecución al script 'entrypoint.sh'.
- Por último, ejecutamos el script 'entrypoint.sh' como comando principal del contenedor.

### ■ Entrypoint:

- Eliminamos la ruta por defecto.
- Detectamos la interfaz de production.net y la establecemos como gateway.
- Establecemos el servidor DNS.
- Configuramos todos los parametros de postfix, el nombre del host, dominio local, indicamos que se usará 'production.com' como origen, hacemos que escuche en todas las interfaces, restringimos solo a IPv4, definimos los dominios locales, configuramos el formato del buzón, etc.
- Creamos archivos de log y otorgamos permisos.
- Configuramos el usuario 'john' estableciendo su shell y creando ciertos directorios.
- Creamos la estructura de carpetas necesarias para el buzón Maildir para 'john' y 'root'.
- Configuramos el cliente 'mutt' para los usuarios 'root' y 'john', haciendo que use Maildir.
- Iniciamos el servidor de correo y lo mantenemos activo.

## 2. Pruebas de funcionamiento

### 2.1. Comprobaciones dentro del contenedor:

Accedemos al contenedor y hacemos una serie de comprobaciones como asegurarnos que el gateway y el dns son los correctos y para comprobar la conectividad haremos varios pings, por ejemplo probamos a hacer ping al dns, que es un contenedor de la subred services y también haremos ping a un contenedor de la red development, por ejemplo al servidor ssh, vemos que ninguno de los pings llegan ya que production esta aislada, aunque esto no quiere decir que no funcione como DNS, el último ping, con destino al servidor Apache, ubicado en la propia red production si que llega, esto es lo correcto según los guiones de las prácticas anteriores, todo lo descrito se ve en la siguiente imagen:

```
john@DebianDocker:~/Escritorio/AS/PRACTICAS$ docker exec -it postfix_mail bash
root@b297e0b3934d:/# cat /etc/resolv.conf
nameserver 172.20.0.6
root@b297e0b3934d:/# ip route
default via 172.30.0.1 dev eth1
172.18.0.0/16 dev eth0 proto kernel scope link src 172.18.0.3
172.30.0.0/24 dev eth1 proto kernel scope link src 172.30.0.15
root@b297e0b3934d:/# ping 172.20.0.6
PING 172.20.0.6 (172.20.0.6) 56(84) bytes of data.
^C
--- 172.20.0.6 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1040ms

root@b297e0b3934d:/# ping 172.40.0.9
PING 172.40.0.9 (172.40.0.9) 56(84) bytes of data.
^C
--- 172.40.0.9 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1022ms

root@b297e0b3934d:/# ping 172.30.0.20
PING 172.30.0.20 (172.30.0.20) 56(84) bytes of data.
64 bytes from 172.30.0.20: icmp_seq=1 ttl=64 time=27.0 ms
64 bytes from 172.30.0.20: icmp_seq=2 ttl=64 time=0.086 ms
^C
--- 172.30.0.20 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.086/13.552/27.018/13.466 ms
root@b297e0b3934d:/#
```

Figura 4: Comprobaciones dentro del contenedor postfix\_mail.

## 2.2. Enviar correo:

Accedemos al contenedor y enviaremos un correo de prueba desde el usuario 'root' al usuario 'john' y posteriormente 'john' responderá al correo de 'root'.

Para ello, accedemos al contenedor y desde la terminal(usuário root) ejecutamos el siguiente comando:

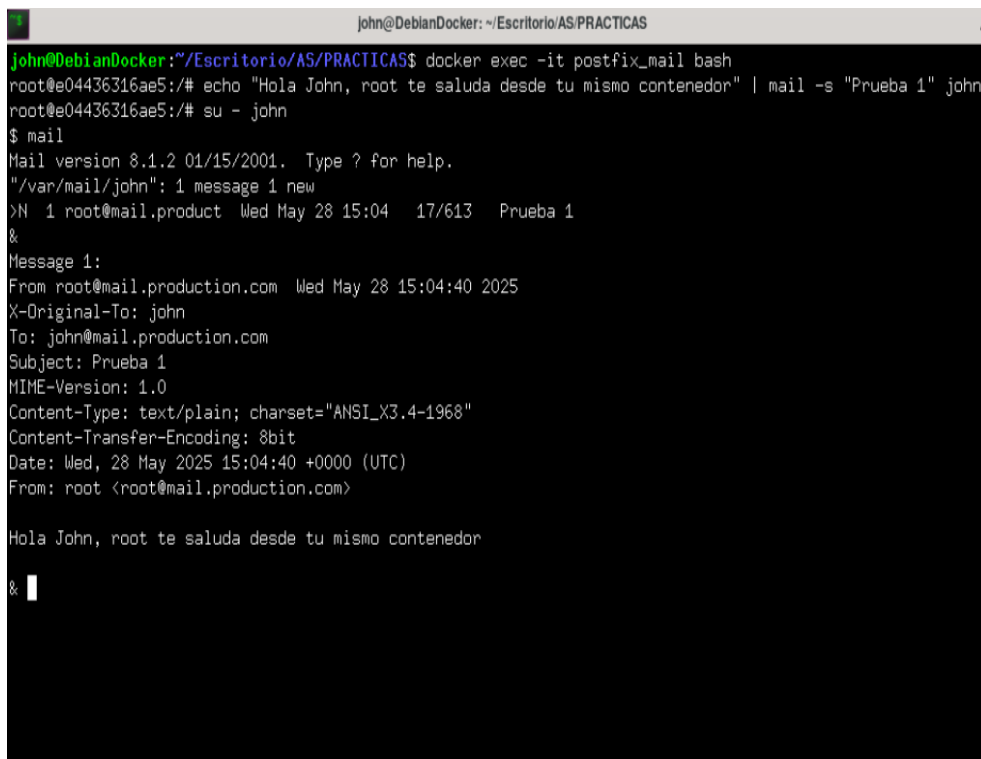
```
echo "Hola John, root te saluda desde tu mismo contenedor"— mail -s "Prueba 1" john
```

Para comprobar ese correo accedemos al usuario 'john' en la terminal y accediendo al mail mediante los siguientes comandos:

```
su - john
```

```
mail
```

A continuación se mostrará el buzón del usuario 'john' y veremos el correo recibido por parte de 'root', el cual podremos visualizar pulsando la tecla 'enter', como podemos ver en la siguiente imagen:



```
john@DebianDocker: ~/Escritorio/AS/PRACTICAS
john@DebianDocker:~/Escritorio/AS/PRACTICAS$ docker exec -it postfix_mail bash
root@e04436316ae5:/# echo "Hola John, root te saluda desde tu mismo contenedor" | mail -s "Prueba 1" john
root@e04436316ae5:/# su - john
$ mail
Mail version 8.1.2 01/15/2001. Type ? for help.
"/var/mail/john": 1 message 1 new
>N 1 root@mail.product Wed May 28 15:04 17/613 Prueba 1
&
Message 1:
From root@mail.production.com Wed May 28 15:04:40 2025
X-Original-To: john
To: john@mail.production.com
Subject: Prueba 1
MIME-Version: 1.0
Content-Type: text/plain; charset="ANSI_X3.4-1968"
Content-Transfer-Encoding: 8bit
Date: Wed, 28 May 2025 15:04:40 +0000 (UTC)
From: root <root@mail.production.com>

Hola John, root te saluda desde tu mismo contenedor
&
```

Figura 5: Usuario 'john' lee el correo enviado por 'root'.

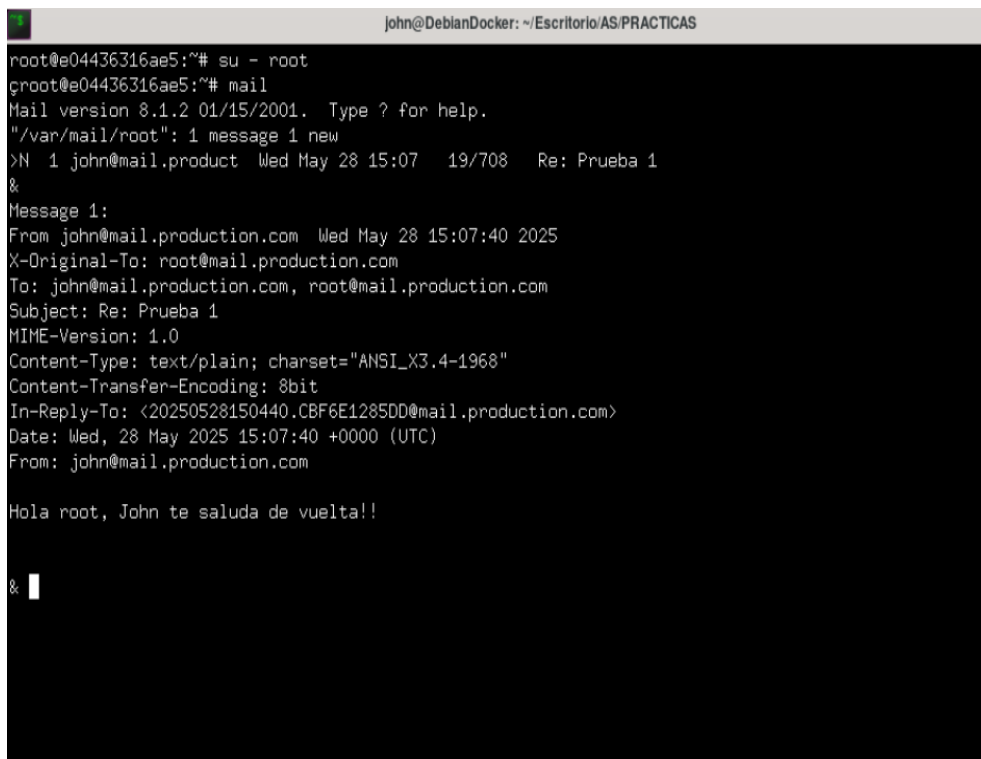
Podremos responderle pulsando la tecla 'r', introducimos el mensaje de respuesta y para enviarlo deberemos pulsar 'enter', escribir un punto y volver a pulsar 'enter', nos pedirá si queremos enviarlo a alguien más, pulsamos 'enter' y por ultimo 'q', para salir del buzón.

Para comprobar que el correo ha llegado de vuelta accedemos al buzón del usuario root mediante los siguientes comandos:

```
su - root
```

```
mail
```

De igual manera que antes, debe aparecer el correo en el buzón y para leerlo pulsamos 'enter', como podemos observar en la siguiente imagen:

A terminal window titled 'john@DebianDocker: ~/Escritorio/AS/PRACTICAS' shows the execution of the 'mail' command as root. The output displays mail version 8.1.2, a list of messages (1 new), and the details of message 1. The message is from john@mail.production.com to root@mail.production.com, with subject 'Re: Prueba 1'. The body of the message says 'Hola root, John te saluda de vuelta!!'. The prompt '>' is visible at the bottom of the message content.

```
john@DebianDocker: ~/Escritorio/AS/PRACTICAS
root@e04436316ae5:~# su - root
root@e04436316ae5:~# mail
Mail version 8.1.2 01/15/2001.  Type ? for help.
"/var/mail/root": 1 message 1 new
>N 1 john@mail.product Wed May 28 15:07  19/708  Re: Prueba 1
&
Message 1:
From: john@mail.production.com Wed May 28 15:07:40 2025
X-Original-To: root@mail.production.com
To: john@mail.production.com, root@mail.production.com
Subject: Re: Prueba 1
MIME-Version: 1.0
Content-Type: text/plain; charset="ANSI_X3.4-1968"
Content-Transfer-Encoding: 8bit
In-Reply-To: <20250528150440.CBF6E1285DD@mail.production.com>
Date: Wed, 28 May 2025 15:07:40 +0000 (UTC)
From: john@mail.production.com

Hola root, John te saluda de vuelta!!

& █
```

Figura 6: Usuario 'root' ve la respuesta de 'john'.

Como hemos comprobado, podemos enviar correos entre los usuarios 'root' y 'john' lo que quiere decir que la conexión se ha establecido correctamente.